Para qué es esta guía. En la Clase 19 viste la idea: una **variable objetivo**, unas **variables explicativas**, un reparto entre **entrenamiento y prueba** y una **línea base**. Hoy lo haces de verdad, con el modelo más sencillo y más usado: la **regresión lineal**. Esta guía lo hace en Orange, sin escribir código.

El caso: ¿cuántas bicicletas preparar?

Capital Bikeshare alquila bicicletas en Washington D. C. Cada noche decide cuántas bicicletas y cuánto personal tener listos para el día siguiente. Tienes dos años de datos **reales**, día por día (2011 y 2012), con el clima de cada día. La pregunta: **¿cuántas bicicletas se alquilarán en un día, según el clima y el tipo de día?** El objetivo es un número: es un problema de **regresión**.

Dos rutas, los mismos números. Esta práctica tiene dos rutas: esta, en Orange, y otra en Python, con el notebook `lab-regresion-lineal.ipynb`. Las dos usan el mismo archivo y reparten los días entre entrenamiento y prueba con la misma semilla, así que eligen los mismos días y **las cifras de las cajas verdes son idénticas en las dos**. Al final de cada caja verde, la etiqueta **[Python: ...]** dice qué hace la otra ruta: si trabajas con alguien de Python, comparen resultados.

Al terminar deberías tener:

1. Un flujo de trabajo de Orange guardado (`bicicletas.ows`) con los nueve pasos.
2. Tres modelos comparados con días que no vieron: la línea base, una regresión simple y una múltiple.
3. La **ficha de la última página**: tu modelo explicado en lenguaje de negocio.

Antes de empezar

- Instala Orange desde orangedatamining.com/download (es gratuito, para Windows, macOS y Linux).
- Descarga el archivo `bicicletas_diarias.csv`.
- Si tu Orange está en español, los nombres de los **widgets** siguen en inglés (**File**, **Linear Regression**...); las opciones sí aparecen traducidas. En la guía va el nombre en español y, entre paréntesis, el inglés.
- Cómo se arma un flujo: arrastra un widget desde el panel izquierdo al lienzo, y para conectarlo arrastra desde el borde derecho de un widget hasta el borde izquierdo de otro. Doble clic abre un widget.

Los datos y el flujo completo

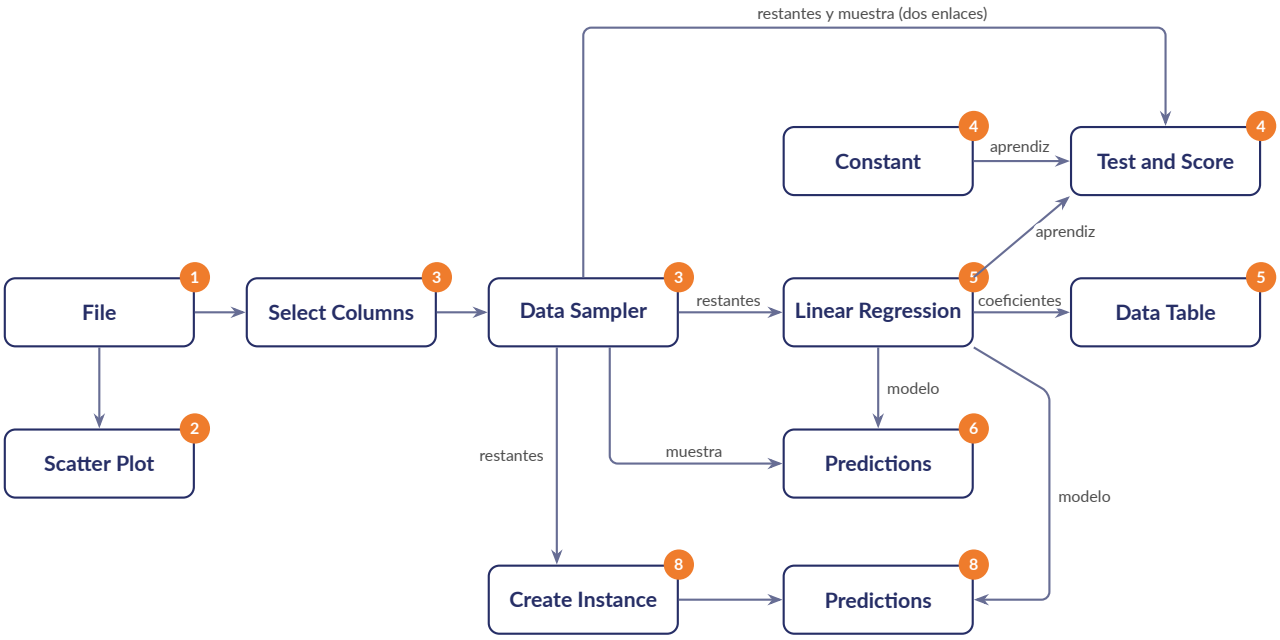
`bicicletas_diarias.csv`: una fila por día, del 1 de enero de 2011 al 31 de diciembre de 2012 (731 días).

Columna	Qué contiene
`fecha`, `anio`, `mes`, `dia_semana`	Cuándo.
`dia_laboral`	1 si es día laboral; 0 si es fin de semana o festivo.
`clima`	Despejado, Nublado o con niebla, Lluvia o nieve.
`temperatura_c`, `sensacion_termica_c`	Temperatura y sensación térmica, en grados Celsius.
`humedad`	Humedad relativa, de 0 a 100.
`viento`	Velocidad del viento, en la escala del archivo original.
`es_2012`	1 si el día es de 2012; 0 si es de 2011.
`lluvia`	1 si hubo lluvia o nieve; 0 si no.
`alquileres`	**Objetivo**: bicicletas alquiladas ese día.

Fuente: Bike Sharing Dataset (Capital Bikeshare), UCI Machine Learning Repository, licencia CC BY 4.0. Fanaee-T, H. y Gama, J. (2013), Event labeling combining ensemble detectors and background knowledge, Progress in Artificial Intelligence. Se tradujeron las columnas y se devolvieron la temperatura, la humedad y el viento a su escala original.

El flujo que vas a construir

Así se verá tu lienzo al final. Los números son los pasos de la guía.



*Las etiquetas de las flechas son las señales que eliges al conectar: "restantes" (el 80 %, entrenamiento) y "muestra" (el 20 %, prueba) son las dos salidas de **Data Sampler**. El paso 7 no agrega widgets: cambia las columnas en **Select Columns**, y el 9 cambia valores en **Create Instance**.*

1 Cargar y revisar los datos

Pregunta: *¿Qué tengo, y hay algo raro?*

1. Arrastra un **File** al lienzo, ábrelo y elige `bicicletas_diarias.csv` con el botón de la carpeta.
2. Abajo, en **Columnas (dé doble clic para editar)** (Columns (Double click to edit)), mira la columna **Tipo** (Type). Orange supone que `es_2012`, `dia_laboral` y `lluvia` son **categórico** (categorical), porque solo tienen 0 y 1. Haz doble clic en el tipo de cada una y cámbialo a **numérico** (numeric). Pulsa **Aplicar** (Apply).
3. Conecta un **Data Table** al **File** y ábrelo. Haz clic en el encabezado `humedad` para ordenar de menor a mayor.

Por qué cambiar el tipo de las columnas de 0 y 1

Si las dejas como categóricas, Orange da dos coeficientes por columna (`es_2012=0` y `es_2012=1`, con el mismo valor y signo contrario) y no uno solo, como Python. Las predicciones serían iguales, pero los coeficientes no se podrían comparar con la otra ruta.

Debes ver

731 filas. La humedad más baja es **0**, el 10 de marzo de 2011. Una humedad relativa de cero no ocurre en una ciudad: es un **error de registro**. Es un solo día entre 731 y casi no mueve el modelo, así que lo dejamos; pero en tu proyecto, cada decisión sobre un dato raro va en la bitácora (Clase 12). **[Python: `dias.describe()` y `dias[dias["humedad"] == 0]`]**

2 Mirar la relación antes de modelar

Pregunta: *¿Los días más cálidos tienen más alquileres? ¿La relación es una recta?*

1. Conecta un **Scatter Plot** al **File**.
2. Eje X: `temperatura_c`. Eje Y: `alquileres`.
3. Marca **Mostrar línea de regresión** (Show regression line).

Debes ver

Una nube que sube de izquierda a derecha: relación **positiva** y moderada (la correlación es **0,627**). Fíjate en los extremos: en los días más calurosos, a la derecha, muchos puntos quedan **por debajo** de la línea. Guarda esa observación para el paso 6. **[Python: `dias.plot.scatter(...)` y `.corr()`]**

3 Elegir columnas y separar entrenamiento y prueba

Pregunta: *¿Con qué aprende el modelo, y con qué lo examino?*

1. Conecta un **Select Columns** al **File**. Deja en **Variables** (Features) solo `temperatura_c` y pon `alquileres` en **Objetivo** (Target). Todo lo demás va a **Ignorado** (Ignored).
2. Conecta un **Data Sampler** al **Select Columns**. En **Tipo de muestreo** (Sampling Type) elige **Proporción fija de datos** (Fixed proportion of data) y mueve la barra a **20 %**. En **Opciones** (Options), deja marcado **Muestreo replicable (determinista)** (Replicable (deterministic) sampling).

Data Sampler aparta al azar el 20 % de los días y tiene dos salidas: **Muestra de datos** (Data Sample), ese 20 %, que será la **prueba**, y **Datos restantes** (Remaining Data), el otro 80 %, que será el **entrenamiento**. Es el mismo reparto de la Clase 19.

Por qué "replicable"

Con esa casilla, Orange usa siempre la misma semilla (42): el reparto sale igual cada vez que abres el flujo, igual para todos tus compañeros e igual al de Python con `random_state=42`. Sin ella, cada vez saldrían otros días y otras cifras.

Debes ver

584 días de entrenamiento y **147** de prueba: lo dice el propio **Data Sampler**, y también aparece al pasar el cursor sobre las flechas que salen de él. [Python: `train_test_split(dias, test_size=0.2, random_state=42)`]

4 La línea base: predecir siempre el promedio

Pregunta: *¿Cuánto se equivoca alguien que no mira ningún dato?*

1. Arrastra un **Constant** (predice siempre el promedio del entrenamiento) y un **Test and Score**.
2. Conecta **Constant** con **Test and Score**.
3. Conecta **Data Sampler** con **Test and Score** y, en la ventana **Editar enlaces** (Edit Links) (doble clic sobre la flecha), arma dos enlaces: **Datos restantes** (Remaining Data) con `Data` (el entrenamiento) y **Muestra de datos** (Data Sample) con **Datos de prueba** (Test Data).
4. En **Test and Score** elige **Probar con datos de prueba** (Test on test data).

Para medir el error, Orange muestra varias columnas. Usaremos tres:

MAE	Error absoluto medio: en promedio, ¿por cuántas bicicletas se equivoca? Está en las mismas unidades del objetivo.
RMSE	Raíz del error cuadrático medio: también en bicicletas, pero castiga más los errores grandes.
R2	Qué parte de la variación de los alquileres explica el modelo. 1 es perfecto; 0 es lo mismo que predecir el promedio.

Debes ver

Fila **Constant**: MAE **1712,0**, RMSE **2022,2** y R2 **-0,020**. Sin mirar ningún dato, nos equivocamos en unas 1700 bicicletas por día: todo modelo tiene que hacerlo mejor. (El R2 sale un poco negativo porque el promedio del entrenamiento, 4561,0, no es exactamente el de la prueba.) Orange muestra más decimales: redondea. [Python: `mean_absolute_error(prueba["alquileres"], pred_base)`]

5 Regresión lineal simple: solo la temperatura

Pregunta: *¿Cuántas bicicletas más por cada grado?*

El modelo busca la recta que mejor sigue a los puntos:

$$\text{alquileres} = \text{intercepto} + \text{pendiente} \times \text{temperatura}$$

1. Arrastra un **Linear Regression**. Déjalo en **Sin regularización** (No regularization), con **Ajustar intercepto** (Fit intercept) marcado.
2. Conecta **Data Sampler** con **Linear Regression**, con el enlace **Datos restantes** (Remaining Data) (el entrenamiento), y **Linear Regression** con **Test and Score**.
3. Para ver la recta, conecta **Linear Regression** con un **Data Table**. En **Editar enlaces** (Edit Links) elige la salida **Coeficientes** (Coefficients).

Debes ver

Coeficientes: `intercept` **1272,76** y `temperatura_c` **160,35**. **Test and Score**, fila **Linear Regression**: MAE **1291,7**, RMSE **1545,9** y R2 **0,404**. **[Python: `LinearRegression().fit(...)`, `.intercept_` y `.coef_`]**

Cómo se lee, en lenguaje de negocio: *por cada grado más de temperatura, se alquilan en promedio unas 160 bicicletas más al día*. El modelo mejora a la línea base (1292 frente a 1712), pero la temperatura sola explica cerca del 40 % de la variación. El intercepto sería el valor a 0 grados, pero ningún día de los datos llegó a 0 (el más frío tuvo 2,4): no lo leas como una predicción.

6 Los errores: lo que el modelo no explica

Pregunta: *¿En qué días se equivoca, y por qué?*

1. Arrastra un **Predictions**. Conecta **Data Sampler** (**Muestra de datos** (Data Sample), la prueba) y el modelo de **Linear Regression**.
2. En **Error de regresión mostrado** (Shown regression error) deja **Diferencia** (Difference). Haz clic en el encabezado de esa columna para ordenar.
3. Vuelve al **Scatter Plot** del paso 2 y mira dónde quedan los puntos respecto a la línea, en los días fríos, templados y calurosos.

El signo de la diferencia

Orange calcula **predicho menos real**: una diferencia positiva es un día en que el modelo predijo de más. En Python, el **residuo** es al revés, real menos predicho. Es la misma cifra con el signo cambiado.

Debes ver

En el **Scatter Plot**, la mayoría de los días fríos (8 de cada 10, por debajo de 10 grados) y de los muy calurosos (7 de cada 10, por encima de 30) quedan **por debajo** de la línea; entre 20 y 30 grados abundan los puntos altos, **por encima**. El modelo predice de más en los extremos y de menos en el medio: la relación tiene forma de **arco**, y una recta no puede doblarse. Es un **límite del modelo lineal**, no un error tuyo. **[Python: residuos por tramos de temperatura: -897, 107, 290 y -923]**

7 Regresión lineal múltiple: más variables explicativas

Pregunta: *¿Cuánto mejora el modelo con el clima y el calendario?*

Con lo que aprendiste en la Clase 19, agrega variables que se conocen **antes** de que pase el día (el pronóstico del clima, el calendario) y que no repiten lo que dice otra. Abre **Select Columns** y pasa a **Variables** (Features): `humedad`, `viento`, `es_2012`, `dia_laboral` y `lluvia`. Todos los widgets se recalculan solos. No uses `sensacion_termica_c`: su correlación con la temperatura es de 0,99, así que dice casi lo mismo.

Debes ver

Test and Score, fila **Linear Regression**: MAE **733,4**, RMSE **919,1** y R2 **0,789**. El error promedio bajó de unas 1700 bicicletas (línea base) a unas 730: **menos de la mitad**. Los coeficientes, en el **Data Table**:

Variable	Coeficiente	En lenguaje de negocio
`intercept`	2070,77	
`temperatura_c`	152,56	Cada grado más, unas 153 bicicletas más, con lo demás igual.
`humedad`	−16,75	Cada punto más de humedad, unas 17 menos.
`viento`	−51,17	Cada punto más de viento, unas 51 menos.
`es_2012`	1959,94	Un día de 2012 tiene unas 1960 más que uno parecido de 2011: el uso del sistema creció.
`dia_laboral`	145,40	Un día laboral tiene unas 145 más: poca diferencia.
`lluvia`	−1239,71	Un día con lluvia o nieve, unas 1240 menos.

[Python: `columnas = [...]` y `LinearRegression().fit(entrenamiento[columnas], ...)`]

"Con lo demás igual" es la clave de la regresión múltiple: cada coeficiente es el efecto de esa columna **manteniendo fijas las otras**. Por eso el de la temperatura bajó de 160 a 153: parte de lo que antes se le atribuía era del año o de la lluvia. Y cuidado: un coeficiente grande no significa una columna más importante; depende de sus unidades.

8 Predecir un día nuevo

Pregunta: *¿Cuántas bicicletas preparar para mañana?*

1. Arrastra un **Create Instance** y conéctale **Data Sampler** (**Datos restantes** (Remaining Data)): muestra una fila con todas las columnas, para que escribas sus valores. Abajo, desmarca **Anexar esta instancia a los datos de entrada** (Append this instance to input data): así sale solo el día nuevo.
2. Escribe: `temperatura_c` 25, `humedad` 60, `viento` 12, `es_2012` 1, `dia_laboral` 1 y `lluvia` 0. Es un día laboral de 2012, templado y sin lluvia.
3. Conecta **Create Instance** con un segundo **Predictions**, y conéctale también el modelo de **Linear Regression**.
4. Ahora cambia `lluvia` a 1: el pronóstico anuncia lluvia.

Debes ver

Unas **6371** bicicletas sin lluvia y unas **5131** con lluvia. El modelo se equivoca en promedio en unas 730: la predicción es una guía para planear, no una cifra exacta. [Python: `multiple.predict(dia_nuevo)`]

9 Los límites del modelo

Pregunta: *¿Puedo usar el modelo para cualquier día?*

Extrapolar es pedirle una predicción fuera del rango de los datos con que aprendió. Los días de entrenamiento van de 2,4 a 35,3 grados.

1. Vuelve al modelo simple: en **Select Columns**, deja solo `temperatura_c` en **Variables** (Features).
2. En **Create Instance**, escribe en la casilla numérica de `temperatura_c` el valor -10 y luego 45. (La barra solo llega hasta los valores de los datos; la casilla acepta cualquiera.)
3. Vuelve al modelo múltiple y, en el **Predictions** de la prueba, ordena por la diferencia: ¿en qué días se equivocó más?

Debes ver

Unas -331 bicicletas a -10 grados (¡un número negativo de alquileres!) y **8488** a 45 grados, aunque en el paso 6 viste que con calor fuerte los alquileres **bajan**. La recta sigue para siempre; la realidad no. **Nunca uses el modelo lejos de los datos con que aprendió.**
El error más grande de la prueba es el **29 de octubre de 2012**: el modelo predijo unas 2980 bicicletas y se alquilaron 22 (diferencia de 2958). Ese día llegó el **huracán Sandy** a la costa este de Estados Unidos: ningún dato del clima diario anticipa una ciudad cerrada. Entre los siguiente es el 29 de junio de 2012, a 34,2 grados: el modelo predijo unas 8006 y se alquilaron 5463. Otra vez, el calor fuerte. **[Python:** `prueba.sort_values("error")`**]**

Guarda el flujo: **Archivo** (File) > **Guardar** (Save), como `bicicletas.ows`.

El flujo resuelto, para comparar

En la carpeta de la clase está `bicicletas-regresion.ows`, con los nueve pasos armados y el modelo múltiple del paso 7. Ábrelo con **Archivo** (File) > **Abrir** (Open). **Luego carga el archivo de datos**: abre el widget **File** y elige `bicicletas_diarias.csv` con el botón de la carpeta. Al cargarlo, el resto del flujo se recalcula solo y muestra las cifras de esta guía. Úsalo para revisar tu trabajo, no para saltarte los pasos.

Entrega: tu modelo en lenguaje de negocio

Es la misma ficha de la ruta Python. Escríbela para alguien de la gerencia, que no sabe qué es un coeficiente.

Pregunta	Tu respuesta
¿Qué decisión apoya el modelo?	
¿Cuánto mejora a la línea base? (MAE de la base y del modelo)	
¿Qué dos coeficientes le contarías a la gerencia, y cómo los dirías?	
¿En qué días se equivoca más, y por qué?	
¿Qué no debe hacerse con este modelo?	

Retos opcionales

1. Agrega `sensacion_termica_c` a las variables del modelo múltiple. ¿Qué les pasa a los coeficientes de la temperatura y de la sensación térmica? ¿Mejora el MAE? Relaciónalo con la pregunta 3 de la selección de características de la Clase 19.
2. Quita `es_2012`. ¿Cuánto empeora el modelo? ¿Qué significa para la empresa que el año pese tanto?

Equivalencias entre las dos rutas

Tarea	Orange	Python
Cargar los datos	**File**	`pd.read_csv`
Separar entrenamiento y prueba	**Data Sampler** (20 %, replicable)	`train_test_split(..., test_size=0.2, random_state=42)`
Elegir objetivo y variables	**Select Columns**	`X = ...[columnas]`, `y = ...["alquileres"]`
Línea base	**Constant**	el promedio de `y_train`
Ajustar el modelo	**Linear Regression**	`LinearRegression().fit(X, y)`
Ver los coeficientes	salida **Coeficientes** (Coefficients)	`.intercept_` y `.coef_`
Medir con la prueba	**Test and Score**	`mean_absolute_error`, `r2_score`
Predecir un día nuevo	**Create Instance** y **Predictions**	`.predict(dia_nuevo)`

Si algo no cuadra

Lo que ves	Causa más probable
Coeficientes `es_2012=0` y `es_2012=1`, con signos contrarios.	No cambiaste el tipo a numérico en **File** (paso 1).
Test and Score pide datos de prueba o da otras cifras.	Falta la conexión **Muestra de datos** (Data Sample) → **Datos de prueba** (Test Data), o no elegiste **Probar con datos de prueba** (Test on test data).
Todas las cifras son distintas de las de la guía.	En **Data Sampler**: no está en 20 %, falta **Muestreo replicable** (Replicable sampling), o los enlaces están cruzados (el entrenamiento es **Datos restantes** (Remaining Data)).
El MAE no es el de la guía.	Sobra o falta una variable en **Variables** (Features): compara con el paso 5 o el 7.

Documentación oficial de los widgets: File, Scatter Plot, Select Columns, Data Sampler, Constant, Linear Regression, Test and Score, Predictions y Create Instance.